

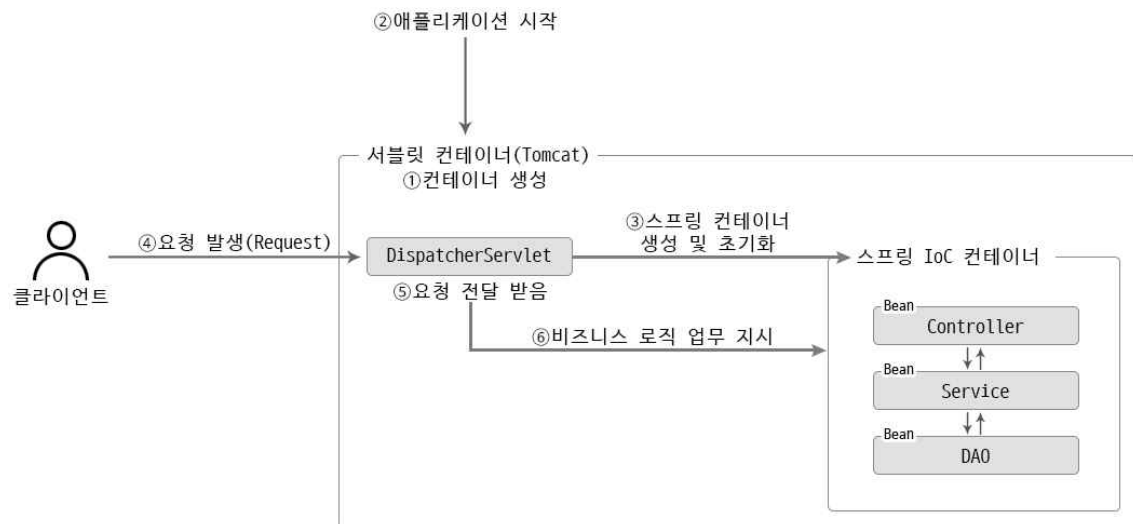
2. Spring Security 구조

Spring Security를 사용하기 위해서는 우선 Spring Security의 기본 구조에 대한 이해가 있어야 합니다. Spring Security의 구조는 사용자 요청(Request)부터 인증 완료까지의 과정을 이해하면 됩니다. Spring Security의 구조에 다소 많은 인터페이스와 객체가 등장합니다. 각각의 역할을 천천히 이해하면서 학습하세요.

2-1. 서블릿 컨테이너(Tomcat)와 스프링 IoC 컨테이너의 관계

서블릿 컨테이너(Tomcat)는 웹 애플리케이션을 실행하는 환경을 제공합니다. 주요 역할은 사용자의 요청을 받아서 서블릿(DispatcherServlet)으로 전달하는 기능입니다. 스프링 IoC(Inversion of Control) 컨테이너는 객체(Bean) 생성 및 의존성 관리가 주요 역할입니다.

서블릿 컨테이너(Tomcat) 안에 Spring IoC 컨테이너가 만들어집니다. ①서블릿 컨테이너(Tomcat)가 먼저 실행된 후, ②스프링 애플리케이션이 구동되면서 ③Spring IoC 컨테이너(ApplicationContext)가 초기화됩니다. 이 과정에서 DispatcherServlet이 서블릿 컨테이너에 등록되며, 이 서블릿이 내부적으로 Spring IoC 컨테이너를 초기화하는 역할을 합니다. 이후 ④사용자 요청이 발생하면 서블릿 컨테이너는 ⑤DispatcherServlet을 이용해서 스프링 IoC 컨테이너의 ⑥컨트롤러를 사용해서 비즈니스 로직을 수행합니다.



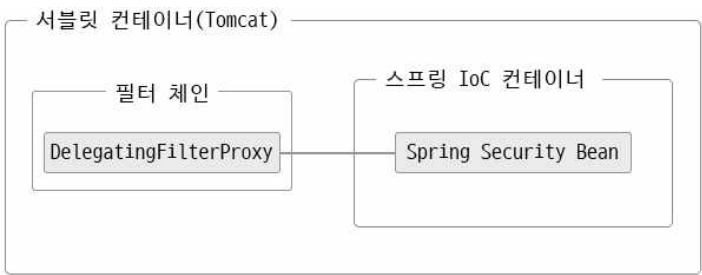
[그림-01] 서블릿 컨테이너(Tomcat)와 스프링 IoC 컨테이너의 관계

위 구조가 입문자한테는 다소 낯설 수 있지만, 중요한 구조이므로 꼼꼼히 이해합니다.

2-2. Spring Security 빈 위치

서블릿 컨테이너(Tomcat)와 스프링 IoC 컨테이너의 관계에 대해서 이해했는데, Spring Security 빈은 어디에 위치하게 될까요? Spring Boot 애플리케이션에 Spring Security를 도입하면 서블릿 컨테이너의 필터 체인에 DelegatingFilterProxy가 등록됩니다. 그리고 Spring Security의 인증과 인가를 담당하는 빈(Bean)은 스프링 IoC 컨테이너에 등록되고 관리됩니다. 결과적으로 Spring Security는 Spring IoC 컨테이너에서 동작하면서, 서블릿 컨테

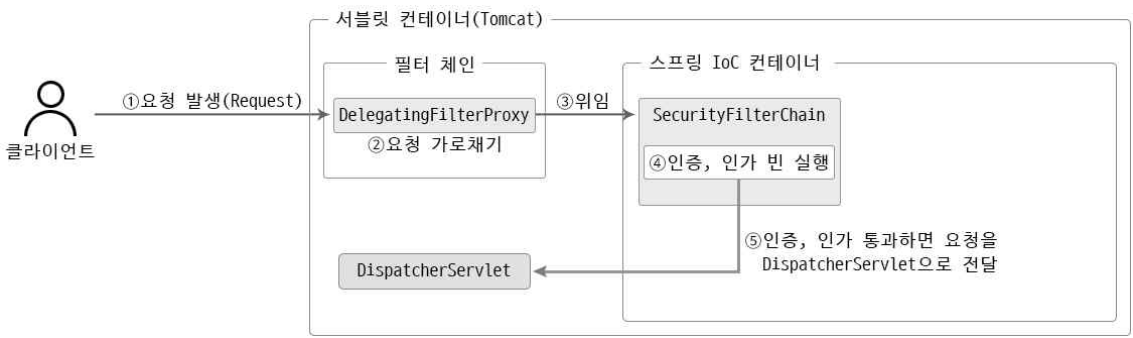
이너의 필터 체인과 연결되는 구조를 만듭니다.



[그림-02] Spring Security는 필터 체인과 연결됨

DelegatingFilterProxy는 서블릿 컨테이너의 필터(Filter)와 Spring Security의 SecurityFilterChain을 연결하는 역할을 합니다. 그리고 서블릿 컨테이너(Tomcat)에서는 일반적인 서블릿 필터처럼 동작하지만, 실제 보안 로직은 Spring IoC 컨테이너에 위임(Delegate)합니다.

지금까지의 내용이 다소 복잡해 보이는데요, 간단하게 정리하면 ①클라이언트 요청이 들어오면 ②DelegatingFilterProxy가 실행(요청 가로채기)됩니다. 하지만 DelegatingFilterProxy는 아무런 보안 로직을 수행하지 않고 Spring IoC 컨테이너의 SecurityFilterChain 빈을 찾아 ③위임합니다. 이후 실제 인증 및 인가는 ④SecurityFilterChain에서 처리합니다. 마지막으로 인증 및 인가를 통과하면 요청은 ⑤DispatcherServlet에 전달되고 Spring MVC의 Controller가 실행되게 됩니다.

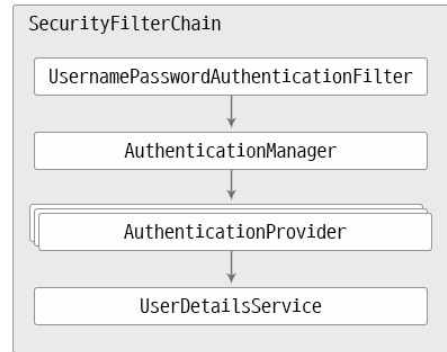


[그림-03] 요청에 대한 Spring Security 실행 구조

2-3. SecurityFilterChain 요청 처리 구조

요청을 위임받은 SecurityFilterChain는 내부적으로 다양한 빈을 이용해서 인증을 진행합니다. 인증에 사용되는 빈으로는

UsernamePasswordAuthenticationFilter,
AuthenticationManager,
AuthenticationProvider,
UserDetailsService 등이 있는데요, 실행 순서와 각각의 역할에 대해서 살펴보겠습니다.



▶ UsernamePasswordAuthenticationFilter: 요청 접수(가로채기) 담당

폼 기반 로그인인 경우 UsernamePasswordAuthenticationFilter가 사용자 요청을 접수(가로채기)하는 역할을 담당합니다. 그리고 Authentication 객체가 만들어지면서 이곳에 사용자 정보(ID, PW)가 저장되고 관리됩니다. UsernamePasswordAuthenticationFilter는 Authentication을 AuthenticationManager에 전달합니다.

▶ AuthenticationManager: 인증 업무 관리 담당

AuthenticationManager는 UsernamePasswordAuthenticationFilter가 전달한 Authentication를 이용해서 전체적인 인증 업무를 관리합니다. AuthenticationManager는 직접 인증 업무를 수행하지 않고 AuthenticationProvider에게 위임하는데요, 이유는 인증을 처리하는 방법이 다양하고 그에 따른 AuthenticationProvider도 다양하기 때문입니다.

‘AuthenticationProvider가 다양하다’는 것은 여러 종류의 AuthenticationProvider가 있다는 뜻으로 데이터베이스 기반 인증에 사용하는 DaoAuthenticationProvider, JWT(JSON Web Token) 기반 인증에 사용하는 JwtAuthenticationProvider, OAuth2 기반 인증에 사용하는 OAuth2LoginAuthenticationProvider 등이 있습니다. 즉, AuthenticationManager는 관리자로 인증 업무에 적합한 AuthenticationProvider를 선택하고 업무를 지시하는 역할을 합니다. AuthenticationManager는 인터페이스로 일반적으로는 이를 구현한 ProviderManager를 사용합니다.

▶ AuthenticationProvider: 인증 담당

AuthenticationProvider는 Authentication에 저장된 사용자 정보(ID, PW)를 검증(비교)하는 역할을 합니다. 이때 AuthenticationProvider가 직접 DB에 저장되어 있는 사용자 정보를 가져오지 않고 UserDetailsService를 이용합니다. AuthenticationProvider는 인터페이스로 대표적인 구현체는 DaoAuthenticationProvider입니다. DaoAuthenticationProvider는 DB의 사용자 정보를 이용해서 인증을 처리합니다.

▶ UserDetailsService: 정보 조회 담당

UserDetailsService는 데이터베이스에서 사용자 정보를 조회합니다. UserDetailsService는 인터페이스로 일반적으로 개발자는 이를 직접 구현한 클래스를 이용합니다.

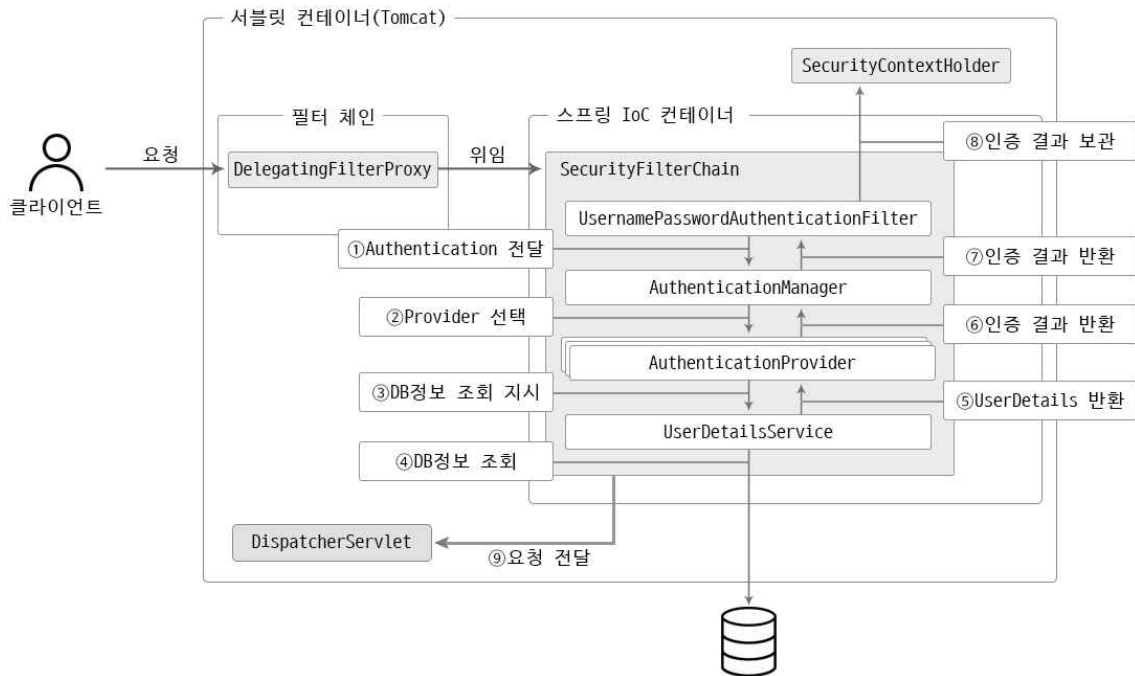
▶ UserDetails: DB에서 조회된 사용자 정보 저장

UserDetailsService가 DB에서 사용자 정보를 조회해서 UserDetails(사용자 정보 포함) 객체를 생성하고 이를 AuthenticationProvider에 반환합니다. UserDetails는 인터페이스로 이를 구현한 클래스로 User를 이용합니다. User 클래스는 Spring Security에서 제공하는 간단한 구현체로, 필요에 따라서 개발자가 UserDetails를 직접 구현한 클래스를 만들어 사용하기도 합니다.

▶ SecurityContextHolder: 인증 정보 보관

사용자가 인증에 성공하면 인증 정보는 Authentication에 저장해서 SecurityContextHolder에 보관됩니다. 이렇게 보관된 정보는 이후 요청 시 재인증 없이도 진행 가능하도록 합니다.

Spring Security를 처음 접하는 입문자의 경우 인증 과정이 다소 많아 부담일 수 있습니다. 다음에 나오는 그림의 번호를 따라가며 천천히 반복해서 익혀 보세요.



[그림-04] 사용자 요청에 대한 인증 처리 과정